Relatório Consolidado de V&V

Verificação integral do módulo **CAN Bus** para ASIL-D

Projeto: Autonomous Emergency Braking (AEB) — UFPE / Stellantis
Validador (cross): Lourenço
Autor original: Renato Fagundes
Escopo: `aeb_can.c`, `aeb_can.h` (Communication / CAN Bus)
Padrão aplicado: ISO 26262-6:2018 · ASIL-D · MISRA C:2012 · CAN 2.0A · DBC
Data: 23 de abril de 2026

Sumário executivo

Este documento consolida as cinco atividades de verificação aplicadas ao módulo CAN Bus durante a cross-validation independente para ASIL-D: testes unitários nominais, cobertura estrutural MC/DC, injeção sistemática de falhas, verificação dinâmica de segurança de memória, e análise estática. O processo atingiu **100% em todas as métricas estruturais** (statement, branch, MC/DC) com a combinação da suíte nominal existente (autoria Renato) e duas suítes complementares produzidas durante a cross-validation: testes estruturais de gaps de cobertura e testes de injeção de falhas. A verificação dinâmica (Valgrind, ASan) não detectou erros de segurança de memória. A injeção sistemática de falhas (31 asserções em 4 categorias) **teve todas as asserções aprovadas**; contudo, o UBSan (`-fsanitize=float-cast-overflow`) revelou **uma vulnerabilidade semântica** na função auxiliar `encode_unsigned`, catalogada neste relatório com o respectivo patch proposto.

1 Visão geral das atividades

A ISO 26262-6:2018 define, nas tabelas 8, 10, 11 e 12 da Parte 6, um conjunto de técnicas de verificação "altamente recomendadas" (++) para ASIL-D. Aplicamos cinco dessas técnicas ao escopo CAN:

Table 1: Panorama das atividades de V&V para o módulo CAN.

#	Atividade	Ferramenta	Norma ISO 26262-6	Resultado (escopo CAN)
1	Testes unitários nominais	Framework próprio (C99)	Tab. 10 item 1b (++)	13 testes · 36 asserções · 100% PASS
2	Cobertura MC/DC	gcc-14 + gcov-14	Tab. 12 item 1c (++)	100% stmt/branch/MC/DC
3	Fault injection	Framework próprio	Tab. 11 item 1e (++)	31 asserções · 31 PASS
4	Memory safety dinâmica	Valgrind + ASan + UBSan	Tab. 8 item 1d (++)	Valgrind/ASan: 0; UBSan: 1 UB
5	Análise estática	cppcheck + MISRA addon	Tab. 8 item 1b (++)	2 advisory em `aeb_can.{c,h}`

Todas as ferramentas são open-source, reproduzíveis em Ubuntu 24.04, e automatizadas pelo novo alvo `make vv-can` adicionado ao Makefile raiz.

1.1 Princípio de independência

A ISO 26262 exige, para ASIL-D, que a verificação de unidade seja conduzida por engenheiro independente do implementador (§5.4.8). A equipe adota a regra de *cross-assignment*: cada membro valida um módulo que não implementou. O módulo CAN, originalmente implementado por **Renato Fagundes**, foi validado independentemente por **Lourenço**. Este relatório documenta integralmente essa validação.

1.2 Delimitação de escopo

Escopo deste relatório vs. MISRA consolidada

A Seção 6 apresenta *análise estática escopada em `aeb_can.c` / `aeb_can.h`*. O **relatório MISRA consolidado do sistema** (cobrindo todos os módulos, requisito do Checklist #8 da Estratégia de Entrega Final) é um entregável separado, produzido pela branch de autoria do Renato (mesma pessoa que é autor original do código CAN aqui validado — os dois papéis são distintos e não criam conflito de independência, pois este relatório valida *apenas* `aeb_can.{c,h}`). Onde esses dois relatórios se sobrepuserem, a fonte-de-verdade para o escopo CAN é este relatório; para o escopo sistêmico, o relatório consolidado.

1.3 Estrutura da suíte de testes

Diferente do UDS (onde a suíte nominal do autor original atingiu 100% de cobertura estrutural por construção), o módulo CAN teve **gaps de cobertura** em ramos defensivos (switch statements) que nenhum teste baseado em requisitos exercitava. Seguindo o mesmo padrão documentado para o módulo PID/Alert, foi acrescentada uma suíte *complementar estrutural* (`test_can_struct.c`, autoria Lourenço) com o mínimo de testes necessários para fechar os gaps. A estrutura final é:

Table 2: Suítes de teste aplicadas ao módulo CAN.

Suíte	Autoria	Casos	Propósito
`test_can.c`	Renato	13	Testes baseados em requisitos (FR-CAN-001..004)
`test_can_struct.c`	Lourenço	13	Fechamento estrutural de ramos defensivos
`test_can_fault.c`	Lourenço	16	Injeção sistemática de falhas

2 Atividade 1 — Testes unitários nominais

A suíte nominal, escrita pelo autor original, cobre os requisitos funcionais **FR-CAN-001** a **FR-CAN-004**. São testes de aceitação por requisito.

Table 3: Distribuição dos testes unitários nominais (escopo CAN, arquivo `test_can.c`).

Grupo de testes	Casos	Requisitos cobertos
Pack/unpack de sinais (round-trip)	1	FR-CAN-003 (DBC signal encoding)
Inicialização (`can_init`)	2	FR-CAN-004 (baud rate 500 kbit/s, falha de HAL)
Decodificação RX AEB_EgoVehicle (0x100)	2	FR-CAN-002, FR-CAN-003
Decodificação RX AEB_RadarTarget (0x120)	1	FR-CAN-002, FR-CAN-003
Decodificação RX AEB_DriverInput (0x101)	1	FR-CAN-002, FR-CAN-003
Timeout de RX (3 × 20 ms)	2	FR-CAN-002 (acceptance: 60 ms)
Transmissão AEB_BrakeCmd (0x080)	1	FR-CAN-003
Transmissão AEB_FSMState (0x200) a 50 ms	1	FR-CAN-001
Transmissão AEB_Alert (0x300)	1	FR-CAN-003
Tratamento de falha de TX (HAL)	1	NFR-SAF-ROB (robustez TX)
Total	**13**	**FR-CAN-001..004, NFR-SAF-ROB**

Resultado global: 13 de 13 casos de teste passaram (**36 asserções, 100% PASS**). Execução via `make test`, integrada à pipeline de CI do GitHub Actions (`ci.yml`).

3 Atividade 2 — Cobertura estrutural MC/DC

Modified Condition/Decision Coverage (MC/DC) é a métrica de cobertura estrutural "altamente recomendada" (++) pela ISO 26262-6 Tab. 12 para ASIL-D. Para cada condição atômica, MC/DC exige demonstrar que, mantendo as demais fixas, alterar apenas ela muda o resultado da decisão — prova de independência lógica.

3.1 Ferramenta utilizada

Utilizou-se o suporte nativo do GCC 14 via flag `-fcondition-coverage` (introduzida em 2024), combinado com `gcov-14 --conditions`. Os dados de cobertura dos três binários de teste (`test_can`, `test_can_struct`, `test_can_fault`) são consolidados via `gcov-tool-14 merge`. Um alvo Makefile `mcdc-can` automatiza o fluxo e grava artefatos em `reports/vv_can/coverage_mcdc/`.

3.2 Análise de gaps e testes complementares

A suíte nominal isolada atinge 89,82% de cobertura de statement para `aeb_can.c`. Os 17 statements não executados concentram-se em dois `switch` statements defensivos (`FSM_state` → `brake_m` e `FSM_state` → `TTC_threshold`) e em ramos defensivos de `NULL`-guard e `CAN_ERR_TX`. Dois esforços complementares, produzidos pelo validador cross (Lourenço), fecharam os gaps:

- `test_can_fault.c` cobriu os ramos de `NULL`-guard e de saturação do `alive_counter`.
- `test_can_struct.c` cobriu cada `case` dos switch defensivos e o ramo `CAN_ERR_TX` de `can_tx_fsm_state`.

3.3 Resultado final

Table 4: Cobertura estrutural do módulo CAN (linha de base + complementares, via `gcov-tool merge`).

Métrica	Valor	Meta ASIL-D	Status
Statement coverage (`aeb_can.c`)	100,00% (167/167)	≥95%	✓ CONFORME
Branch coverage (`aeb_can.c`)	100,00% (81/81)	≥95%	✓ CONFORME
MC/DC coverage (condition outcomes)	100,00% (68/68)	≥95%	✓ CONFORME
Calls exercised	100,00% (41/41)	—	—

Observação de implementação

O módulo CAN contém duas áreas de alta complexidade ciclomática: o `switch` sobre `fsm_state` em `can_tx_brake_cmd` (7 `case` + `default`) e em `can_tx_fsm_state` (3 `case` + `default`). A suíte nominal do autor só exercitava dois desses cases; a suíte estrutural complementar é o que torna a cobertura 100%. Isto mantém a **intencionalidade do autor original** preservada (a suíte dele continua sendo a *fonte de verdade* para requisitos) e adiciona, por fora, a varredura estrutural que a ISO 26262 exige para ASIL-D.

Reprodução: `make mcdc-can` gera `run_{nominal,fault,struct}.log`, `aeb_can.c.gcov` e relatório HTML em `reports/vv_can/coverage_mcdc/`.

4 Atividade 3 — Fault injection sistemática

Fault injection alimenta o código com entradas inválidas, corrompidas ou extremas para verificar que o comportamento permanece fail-safe. É "altamente recomendada" (++) pela ISO 26262-6 Tab. 11 item 1e para ASIL-D.

4.1 Categorias exercitadas

Table 5: Categorias de falha aplicadas ao módulo CAN.

Cat.	Natureza	Exemplos aplicados
A	Valores inválidos	Ponteiros `NULL` em todas as APIs públicas; DLC inferior ao esperado por ID; ID desconhecido; falha forçada do HAL de init (15 asserções)
B	Extremos numéricos	`NaN`, $\pm\infty$, `FLT_MAX`, overflow do cast em `encode_unsigned`; `raw_value` excedendo a largura do sinal em `can_pack_signal` (7 asserções)
C	Estado corrompido (SEU)	`alive_counter` = 0xAA; `rx_miss_count` = 0xFF; `tx_cycle_counter` = `UINT32_MAX`; `initialised` = 0x55 (5 asserções)
D	Persistência	Ciclo timeout → RX válido → timeout; 100 toggles de `force_send_fail`; burst TX > 32 frames (4 asserções)
E	Temporização	**Fora de escopo** no nível unitário (bancada host sem scheduler); diferido para SIL

4.2 Predicado de fail-safe

A predicate de "fail-safe" usada nas categorias A e B é: *sob entrada anormal, a função afetada deve (i) retornar um código de erro explícito (`CAN_ERR_TX` ou `CAN_ERR_INIT`), ou (ii) produzir um frame CAN em que o valor bruto empacotado esteja contido dentro da largura declarada do sinal, sem corromper sinais adjacentes.* Qualquer comportamento que dereferencie `NULL`, escreva bits fora da janela do sinal, ou transmita bytes implementation-defined **não é fail-safe**.

4.3 Resultado

Table 6: Resultado fault injection, módulo CAN.

Categoria	Testes	Asserções	Passaram	Falharam	Veredito
A – Valores inválidos	2	15	15	0	Robusto
B – Extremos numéricos	1	7	7	0	Por coincidência (ver nota)
C – SEU / estado	1	5	5	0	Robusto
D – Persistência	1	4	4	0	Robusto
TOTAL	**5**	**31**	**31**	**0**	—

Nota sobre a Categoria B (extremos numéricos)

As sete asserções de `NaN`/$\pm\infty$/`FLT_MAX` passaram o predicado de fail-safe, mas *por coincidência do comportamento do GCC 14 em x86-64*: o cast `(uint32_t)NaN` e `(uint32_t)`$\pm\infty$ produz 0 nessa toolchain/plataforma. O padrão C11 §6.3.1.4 classifica esses casts como comportamento indefinido, e o UBSan (com `-fsanitize=float-cast-overflow`) confirma isso na linha 89 (Seção 5). **A robustez observada não é garantida** ao mudar compilador, target ou nível de otimização. O patch do Bug #1 (Seção 7) torna o comportamento fail-safe por projeto, eliminando a dependência do acaso.

Reprodução: `make fault-can` grava `reports/vv_can/fault_injection/run.log`.

5 Atividade 4 — Memory safety

Verificação dinâmica de segurança de memória usando três ferramentas complementares, aplicadas às suítes nominal e de fault injection do módulo CAN.

Table 7: Ferramentas de verificação dinâmica.

Ferramenta	Classes detectadas	Mecanismo
Valgrind 3.22	Memory leaks, uso de memória não inicializada, acesso fora de limites de blocos alocados	Interpretação em VM
AddressSanitizer	Heap/stack/global buffer overflow, use-after-free, use-after-scope, double-free	Instrumentação em compile-time
UBSanitizer	Signed integer overflow, division by zero, invalid shift, misaligned pointer, **float-cast-overflow**	Instrumentação em compile-time

5.1 Resultado

Table 8: Resultado das verificações dinâmicas (suítes CAN).

Binário	Valgrind	ASan	UBSan (float-cast-overflow)
test_can (nominal)	0 erros	0 erros	0 erros
test_can_fault	0 erros	0 erros	**1 local de UB detectado**

Listing 1: Diagnóstico UBSan encontrado na suíte de fault injection.

```
src/communication/aeb_can.c:89:32: runtime error: nan is outside the
   range of
    representable values of type 'unsigned int'
```

UBSan confirma independentemente o que a Categoria B do fault injection já sugeria por análise: o cast `(uint32_t)(raw_f + 0.5F)` na função auxiliar `encode_unsigned` é comportamento indefinido quando `raw_f` é `NaN`. As chamadas afetadas são todas as encodings DBC de campos float na transmissão (pressão de freio, ângulo TTC, acelerações).

Valgrind e ASan limpos: o código não acessa memória indevidamente, não vaza, e não usa memória não-inicializada. A vulnerabilidade encontrada é **semântica** (cast UB), não de memória.

Reprodução: `make memory-can` grava `valgrind_{nominal,fault}.log` e `ubsan_{nominal,fault}.l` em `reports/vv_can/memory_safety/`.

6 Atividade 5 — Análise estática

A análise estática é exigida pela Tab. 8 item 1b da ISO 26262-6 (uso de subconjunto da linguagem — MISRA C:2012 é o padrão de-facto no setor automotivo).

6.1 Ferramenta e escopo

Utilizou-se o `cppcheck 2.13` com o addon MISRA, via alvo Makefile `misra-can`. O escopo é estritamente `src/communication/aeb_can.c` e `include/aeb_can.h`. A análise

também inclui os cabeçalhos transitivamente incluídos (`aeb_types.h`, `aeb_config.h`) mas os achados nesses arquivos pertencem ao escopo da *análise MISRA consolidada do sistema* (não este relatório; ver Seção 1.2).

6.2 Resultado

Table 9: Resultado cppcheck+MISRA (apenas escopo CAN, `aeb_can.{c,h}`).

Severidade	Issues	Comentário
error	0	Nenhum erro semântico
warning	0	Nenhum comportamento suspeito
performance	0	Nenhum antipadrão
portability	0	Nenhum uso não-portável
style	2	Advisory — ver detalhe abaixo
Total escopo CAN	**2**	**Apenas advisories, sem high/medium**

6.3 Detalhe das advisories in-scope

- `aeb_can.c:313` — MISRA C:2012 **Rule 10.6** (advisory, style): `uint32_t brake_req = (pid_out->brake_pct > 0.0F) ? 1U : 0U;` A expressão ternária é classificada pela regra como uma composite expression atribuída a tipo mais largo que o tipo essencial. **Decisão de justificar** em vez de modificar: o comportamento está correto e a mudança de forma (e.g. via variável intermediária tipada) agregaria verbosidade sem benefício funcional.

- `aeb_can.h:45` — MISRA C:2012 **Rule 2.5** (advisory, style): macro `CAN_OK` (e parceiras `CAN_ERR_*`) marcada como possivelmente não usada dentro do escopo do header. Na prática são constantes públicas usadas por múltiplos módulos e pelos testes, mas o cppcheck não consegue verificar isso sem a análise global.

Para transparência de processo, a varredura completa também visita os cabeçalhos transitivamente incluídos. Foram reportados **60 diagnósticos *style*** nesses cabeçalhos compartilhados (`aeb_types.h` e `aeb_config.h`), todos do grupo MISRA 2.3/2.4/2.5 — regras *advisory* sobre declarações não-utilizadas, atribuíveis à estrutura superset dos cabeçalhos do sistema. Serão tratados no relatório consolidado MISRA sistêmico.

Conclusão: o módulo CAN não contém erros, warnings, performance ou portability issues MISRA dentro do seu escopo de código próprio. As duas advisories de style têm justificativa documentada e não impedem a certificação. Zero violações high/medium.

7 Bug encontrado e patch proposto

7.1 Bug #1 — Cast float→inteiro sem validação em `encode_unsigned`

Localização: `src/communication/aeb_can.c:89`

Descrição: a função auxiliar `encode_unsigned` converte um valor físico (`float32_t`) em valor bruto (`uint32_t`) para empacotamento DBC. O código atual:

Listing 2: Código atual (vulnerável).

```
static uint32_t encode_unsigned(float32_t physical,
                                float32_t factor,
                                float32_t offset)
{
    float32_t raw_f = (physical - offset) / factor;
    uint32_t  raw   = 0U;

    if (raw_f < 0.0F) {
        raw = 0U;
    } else {
        raw = (uint32_t)(raw_f + 0.5F);  /* <-- UB se raw_f = NaN ou
            Inf */
    }
    return raw;
}
```

Problema: se `physical` for `NaN`, a comparação `raw_f < 0.0F` é sempre `false` (regra IEEE 754), portanto o fluxo vai para o `else` e executa `(uint32_t)NaN`, que é *undefined behaviour* segundo C11 §6.3.1.4. O mesmo vale para `physical = +∞`: `raw_f` é `+∞`, `raw_f + 0.5F` é `+∞`, e o cast é UB. Para `physical = FLT_MAX`, o valor de `raw_f` pode exceder `UINT32_MAX` e o cast é UB.

Por que ainda funciona em produção: GCC 14 em x86-64 materializa todos esses UB como `0`, que por acaso é um valor *fail-safe* (freio zero, TTC zero). **Isto não é garantido** em outra toolchain, nível de otimização, ou processador alvo (ARM Cortex).

Impacto: em compilação diferente, o frame CAN AEB_BrakeCmd ou AEB_FSMState pode ser transmitido com um valor bruto arbitrário (lixo de registro FPU). O ECU de freio a jusante interpretaria isso como pressão hidráulica aleatória — potencial frenagem não comandada.

Patch proposto

Listing 3: Normalização explícita de NaN/Inf e saturação.

```
#include <math.h>
#include <stdint.h>

static uint32_t encode_unsigned(float32_t physical,
                                float32_t factor,
                                float32_t offset)
{
    uint32_t raw = 0U;

    /* Guard NaN and +/-Inf per C11 6.3.1.4 */
    if (isnan(physical) || isinf(physical)) {
        raw = 0U;
    } else {
        float32_t raw_f = (physical - offset) / factor;
        if (raw_f < 0.0F) {
            raw = 0U;
        } else if (raw_f > (float32_t)UINT32_MAX) {
```

```
            raw = UINT32_MAX;                /* Saturate to max uint */
        } else {
            raw = (uint32_t)(raw_f + 0.5F);  /* safe path */
        }
    }
    return raw;
}
```

Teste de aceitação: o binário `test_can_fault_san` (ASan+UBSan) deve executar com `0 runtime error`. Os 31 asserções da suíte de fault injection devem continuar passando, e os 13 testes nominais do Renato devem continuar passando sem modificação.

8 Plano de ação e cronograma

Table 10: Ações necessárias para fechar a V&V do CAN.

#	Ação	Responsável	Esforço
1	Aplicar patch do Bug #1 em `aeb_can.c:89` (`encode_unsigned`)	Renato	1–2 h
2	Re-executar `make vv-can` e confirmar UBSan 0 runtime error	Lourenço	10 min
3	Confirmar que 13 nominais + 13 estruturais + 31 fault continuam 100% PASS	Lourenço	5 min
4	Abrir PR com o patch em `fix/can-encode-robustness`, base `development`	Renato	30 min
5	Preencher linhas FR-CAN-* do Excel Funcional	Lourenço	1 h
6	Abrir issue sistêmica para validação NaN/Inf em outros encoders float (PID, TTC)	Lourenço / Renato	30 min

8.1 Narrativa para a defesa

> *"Aplicando a stack completa de verificação para ASIL-D ao módulo CAN, medimos cobertura estrutural 100% (statement, branch, MC/DC) com o auxílio de duas suítes complementares — estrutural e de fault injection — à suíte nominal do autor original. Valgrind e AddressSanitizer não encontraram erros de memória. A injeção sistemática de falhas — 31 asserções em 4 categorias — passou todas as asserções sob a toolchain alvo, mas a categoria de extremos numéricos só é fail-safe por coincidência. UBSan (`-fsanitize=float-cast-overflow`) confirmou independentemente uma vulnerabilidade de cast float→inteiro em `encode_unsigned`. O patch proposto transforma comportamento fail-safe por coincidência da toolchain em fail-safe por projeto. O ciclo completo detecção → correção → re-validação independente exemplifica a aplicação prática do princípio de independência da ISO 26262-6 §5.4.8."*

9 Rastreabilidade: requisito → teste → evidência

Table 11: Sumário de rastreabilidade por requisito. Detalhe completo (uma linha por teste) em `functional_tests_can.csv` — fonte para a aba CAN do Excel Funcional.

Requisito	# Testes	Evidência principal	P/F
FR-CAN-001 Ego dynamics TX (50 ms)	2	`test_can.test_tx_fsm_period` + struct HAL-fail	P
FR-CAN-002 Radar data RX + timeout	4	`test_rx_radar_target`, `test_rx_timeout*`	P
FR-CAN-003 DBC signal encoding	6	`test_signal_roundtrip`, `test_tx_brake_cmd`, `test_tx_alert`, `test_rx_ego_vehicle`, `test_rx_driver_input`, `test_rx_yaw_rate`	P
FR-CAN-004 Baud rate 500 kbit/s	2	`test_init_success`, `test_init_failure`	P
NFR-SAF-ROB Robustez	31	`fault_injection/run.log` + UBSan	**F** (Bug #1)

10 Conformidade com ISO 26262 ASIL-D

Table 12: Mapeamento explícito das atividades → tabelas normativas ISO 26262-6:2018.

Tabela	Item	Técnica exigida	ASIL-D	Aplicada por
8	1b	Uso de subconjunto da linguagem	++	MISRA C:2012 via cppcheck (Seção 6)
8	1d	Implementação defensiva	++	Valgrind + ASan + UBSan (Seção 5)
8	1e	Uso restrito de memória dinâmica	++	Valgrind: 0 bytes heap
10	1b	Testes baseados em requisitos	++	`test_can.c` (13 testes FR-CAN-*)
11	1e	Injeção de falhas / error guessing	++	`test_can_fault.c` (31 asserções, 4 categorias)
12	1a	Statement coverage	++	gcov-14 → 100%
12	1b	Branch coverage	++	gcov-14 → 100%
12	1c	MC/DC coverage	++	gcc-14 `-fcondition-coverage` → 100%

Conclusão de conformidade: todas as 8 técnicas estruturais marcadas como "altamente recomendadas" (++) para ASIL-D nas tabelas 8, 10, 11 e 12 da ISO 26262-6 foram aplicadas ao módulo CAN, com evidência documentada e reproduzível via `make vv-can`. A única não-conformidade encontrada (Bug #1) está rastreada, com patch proposto e teste de aceitação definido.

Referências

- ISO 26262-6:2018 — Road vehicles — Functional safety — Part 6: Product development at the software level. §5.4.8 (independence), Tables 8, 10, 11, 12.
- ISO 11898-1:2015 — Road vehicles — Controller area network (CAN) — Part 1: Data link layer and physical signalling.
- MISRA C:2012, 3rd edition (2019). Rules 2.3/2.4/2.5, 10.6 (advisory; shared-header and ternary scope).
- IEEE 754-2019 — Standard for Floating-Point Arithmetic. Comportamento de NaN em operações aritméticas e comparações.
- ISO/IEC 9899:2011 (C11) §6.3.1.4 — Real floating to integer conversions.
- GCC 14 Release Notes — Condition coverage support (`-fcondition-coverage`).
- Valgrind Developers. Valgrind User Manual, version 3.22.
- Clang/LLVM Team. `-fsanitize=float-cast-overflow` documentation.
- Ammann, P. & Offutt, J. (2016). Introduction to Software Testing, 2nd ed.
- AEB-Stellantis Project — `final_delivery_strategy.tex`, abril 2026.
- Fagundes, R. — `Relatorio_Consolidado_VV_UDS.tex`, abril 2026 (template de referência).

Relatório produzido como parte da cross-validation independente da Deliverable 4/4 do projeto AEB — Residência Tecnológica em Desenvolvimento de Software Embarcado para o Setor Automotivo · UFPE / Stellantis · 2026. Artefatos reproduzíveis via `make vv-can`. Commit: `${GIT_SHA}` (preencher no tag de release).